Rock-Paper-Scissors Game with Machine Learning Algorithm and JavaFX GUI

Assignment 05: Report

San Jose State University

Department of Computer Science

CS 151: Object-Oriented Design

Professor Robert Nicholson

April 08, 2026

Group: Objectively_Outstanding_ Peoples

Submitted by:

- Andrew Dover
- Daniel Chin
- Delis Santiago
- Jan David Ella

1. Introduction

The project presents a modernized version of the traditional Rock-Paper-Scissors game which our team created using Java and JavaFX. The primary goal of Assignment 5 required teams to convert the command-line application from Assignment 4 into a complete interactive graphical user interface (GUI) system while maintaining proper software design practices.

The application uses the Strategy Pattern to create various methods for selecting computer moves, which it combines with its graphical user interface (GUI). The system includes two different strategies that enable players to select between a random strategy and an intelligent strategy that uses past player movements to forecast future strategies. The demonstration shows how software design patterns enhance system adaptability while making system maintenance tasks easier.

2. Events Generated by the Application

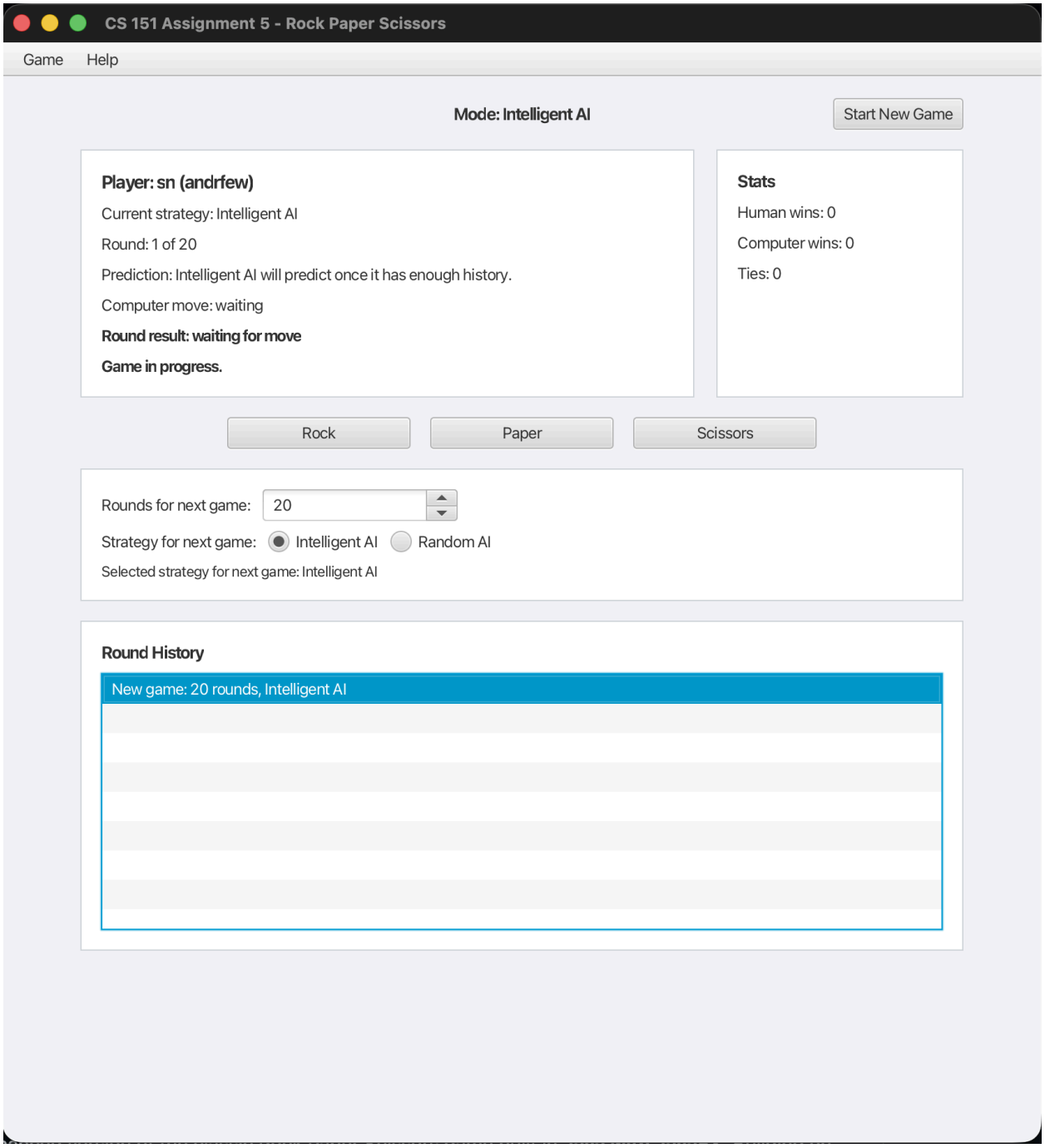
The system activates different functions that respond to user activities because it operates as an event-driven application. The JavaFX interface generates the main events that drive the system.

A user activates the system when he or she selects one of the available buttons, which include Rock, Paper, and Scissors. The controller manages the event that follows, and then it assesses the user's selection before executing the suitable strategy to determine the computer's response.

The game state resets and all round history gets deleted, and all statistics return to their starting point when a player chooses a new game through the menu. The application uses UI events, which include button hover effects and menu selections, to create responsive behavior.

The system processes each event through its established pathway, which begins with user input and then moves to controller functions and strategy implementation before concluding with UI updates that show results.

Figure 1: Example of round history displayed in the application UI



As shown in Figure 1, the system displays the predicted move, current statistics, and round history in real time.

3. Use of Callback Functions

The application uses JavaFX event handlers to execute callback functions throughout its entire structure.

The system executes a callback function whenever users interact with the interface by performing actions such as button clicks. The system uses these callbacks to determine the appropriate actions that should follow each specific event.

The move button activates a callback that performs three tasks. First, the system stores the user's current choice. The system then executes the chosen strategy to create the computer's next move. The system proceeds to present the outcome and relevant data through the user interface.

This method enables the application to maintain both its active performance and its ability to function as separate components. The program operates in a dormant state because it monitors input through event detection, which triggers specified functions when events take place.

4. Code Reuse from Assignment 4

The project reused most of the fundamental logic from Assignment 4. This includes:

- The main game logic that determines the winner.
- The implementation of the Strategy Pattern.
- The strategy pattern incorporates both random and intelligent strategies.
- Data structures used to track user move history.

The team integrated existing logic into the new JavaFX-based application with minimal modifications, primarily adapting the logic to integrate with the JavaFX graphical interface. Furthermore, the team created two distinct systems, which included backend operations and user interface components.

The team showcased effective engineering practices by utilizing previously validated existing components instead of developing new ones.

5. Inversion of Control

The application demonstrates Inversion of Control (IoC) through its implementation of JavaFX together with the strategy pattern.

In a traditional program, the developer controls the flow of execution. However, the application uses inverted control in this situation:

- The framework of JavaFX decides when events should take place.
- The application includes callback functions that the framework will execute at determined times.

For example:

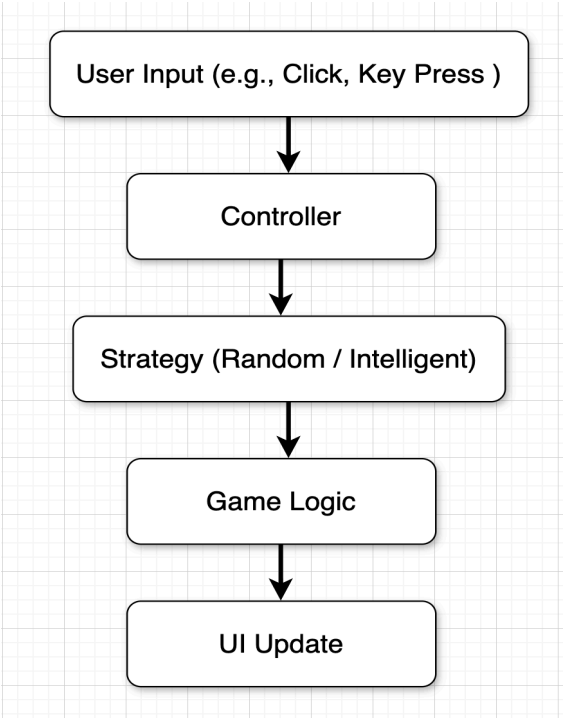
- The program does not manually verify if a button is clicked.
- JavaFX identifies the event and directs it to the appropriate handler.

The Strategy Pattern enables IoC through its ability to let developers inject game decision-making processes during runtime. The strategy interface only needs to know how to call a method; the main game doesn't need to know how the strategy works.

The system creates an architectural framework that enables users to develop their applications without strict dependencies between software components. This separation of control improves modularity, making the system easier to extend, test, and maintain.

The following diagram shows how control flows through the application. The system uses multiple components to manage different user-driven operations instead of one component that controls all system functions.

Figure 2: Control flow of the application demonstrating event-driven design and strategy delegation



As shown in Figure 2, while interaction between a user's input and the logic of the game is coordinated by the controller, the selection of a strategy is delegated to the user.

6. Conclusion

Our team developed a graphical application for Rock-Paper-Scissors using JavaFX, which functions through user events. The code becomes more modular and expandable through the implementation of design patterns, which include the Strategy Pattern.

The application used callback functions and event handling to show how modern applications handle user interactions. The team built a clean and maintainable architectural design through their implementation of code reuse and inversion of control.

Overall, the project demonstrates how graphical user interfaces and software engineering principles must work together to create scalable, maintainable, and user-friendly applications.